Inteligencia artificial - Practica 3

[Elias Agustin Mihailov]
NIP: [927656]

1. Introducción y objetivos

Esta memoria recoge las conclusiones y decisiones de código de la práctica 3 de inteligencia artificial.

La práctica explora algoritmos de búsqueda local, los cuales no buscan obtener un camino óptimo desde un estado inicial al final, sino que solo buscan encontrar dicho estado final lo más rápido posible, sin sacrificar la eficiencia en el proceso. Estos algoritmos parten desde un estado aleatorio del problema e intentan obtener su solución, pero por su implementación, no siempre tienen éxito, al contrario que los algoritmos de búsqueda (si tuvieran el suficiente tiempo).

2. Problema de las N-Reinas

Para las NReinas se ha adaptado la clase original, que estaba pensada para encontrar solución con algoritmos de búsqueda que colocan reinas secuencialmente, para que empiece con un estado aleatorio del problema y que los algoritmos locales encuentren su solución. Se modificaron las funciones heredadas de result(), actions() y value().

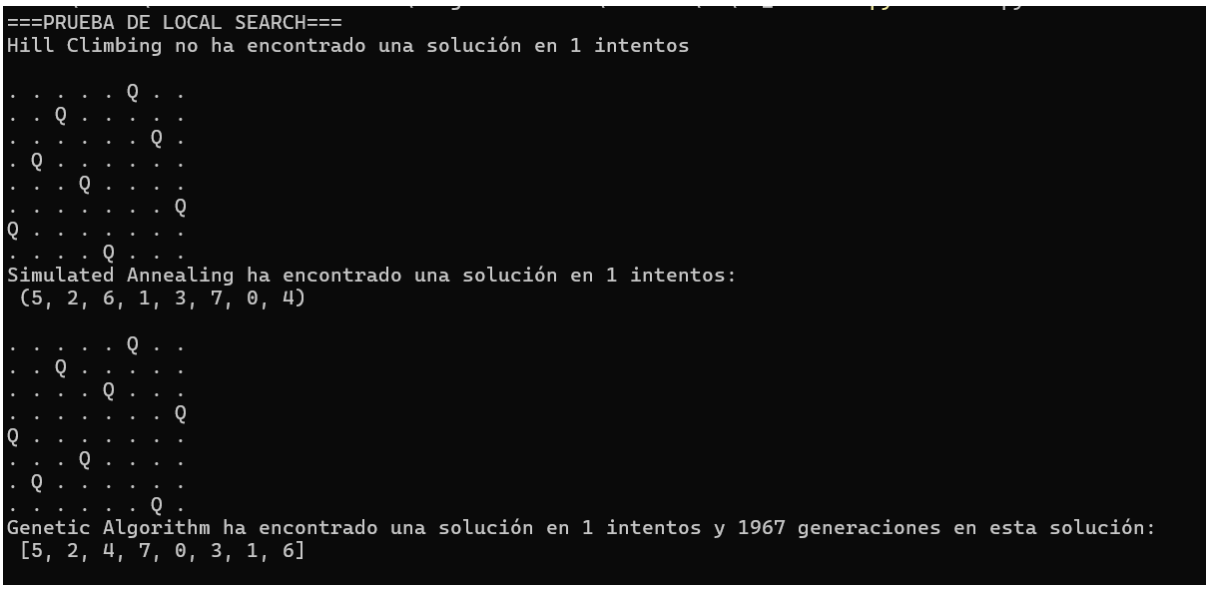
Además en mi caso permito que al inicializar el problema, se le pueda pasar por parámetro una tupla con las reinas colocadas como se desee, sin que sea siempre todo aleatorio.

2.1. Los experimentos y ejecuciones de los distintos algoritmos

```
===PRUEBA DE LOCAL SEARCH===
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . . . . . Q
. . . . Q . . .
Q . . . . . . .
. . . Q . . . .
Hill Climbing ha encontrado una solución en 1 intentos:
 (5, 2, 6, 1, 7, 4, 0, 3)

. . Q . . . . .
. . . . Q . . .
. . . . . . . Q
. . . Q . . . .
Q . . . . . . .
. . . . . Q . .
. Q . . . . . .
. . . . . Q . .
Simulated Annealing ha encontrado una solución en 1 intentos:
 (2, 4, 7, 3, 0, 6, 1, 5)

. . Q . . . . .
. . . . Q . . .
. Q . . . . . .
. . . . . . . Q
Q . . . . . . .
. . . . . Q . .
. . . Q . . . .
. . . . . Q . .
Genetic Algorithm ha encontrado una solución en 1 intentos y 1201 generaciones en esta solución:
 [2, 4, 1, 7, 0, 6, 3, 5]
```



En estas (dos) trazas se encuentran la ejecución individual, de 1 solo intento de cada algoritmo:

- Hill Climbing: Es un algoritmo que siempre busca cual es la mejor acción que puede tomar en un cierto momento, teniendo en cada estado del problema un valor asociado (con una cierta función designada, en este caso parejas de reinas en conflicto), y escoge la acción que le lleve al estado con mayor valor.
  Hill Climbing tiene el problema de encontrarse ante ante estados que llegan a un punto conocido como mínimo local (siempre de la función usada para asignar valor a los estados), en el que cualquier otra acción disminuye su valor, y por tanto no encuentra ninguna acción a realizar y se atasca. Hay varios mecanismos que se usan para solventarlo, como iniciar de nuevo el problema desde otro estado inicial, o como el siguiente algoritmo a discutir, permitir realizar acciones que vayan a estados con menor valor (sea menor al máximo, como directamente acciones que llevan a estados peores que el actual)
- Enfriamiento Simulado: Variante del Hill Climbing, que en vez de buscar la mejor acción, selecciona una aleatoria de entre las posibles. Dicha acción puede tener dos resultados: si es una acción positiva (el siguiente estado cuenta con mayor valor que el actual), se acepta directamente y actúa como un Hill Climbing con una velocidad menor (haría más acciones para llegar al estado final), pero si es una acción negativa, con una cierta probabilidad se acepta y si no se rechaza. Por tanto es vital que dicha probabilidad decrece, para que al final de la ejecución del algoritmo, que es cuando no hay más acciones a realizar para un cierto estado(no ocurre en las NReinas), o cuando el *scheduler de la temperatura* llegue al número de iteraciones límite, la probabilidad de realizar una acción peor (suponiendo que estamos en el estado final, toda acción realizada es peor), sea lo más mínima posible.
- Algoritmo Genético: El algoritmo que se contrapone al Hill-Climbing, que busca rapidez a costa de no encontrar siempre la solución. Los genéticos buscan encontrar una solución creando de manera progresiva una solución (basada en propuestas de solución anteriores). Tiene muchas capas, ya que los genes se cogen de manera aleatoria por

trozos de diferentes “individuos”(propuestas de solución anteriores), a los cuales además se les pueda mutar diferentes partes de su estado para otorgar “atajos” al algoritmos si esas mutaciones tienen éxito.

El algoritmo genera muchas generaciones de soluciones, pero si encuentra en algún momento una solución verdadera al problema, deja de buscar. Teniendo eso en cuenta vemos que el número de generaciones es brutal, ya que es bastante aleatorio encontrar una solución, aunque en principio el algoritmo genera progresivamente mejores resultados. Y con tantas generaciones, el algoritmo casi siempre encuentra una solución.

Por tanto, vemos que las trazas pueden representar la explicación anterior, al algoritmo genético ha encontrado solución (casi siempre lo hace, a costa de más generaciones y tiempo de ejecución), mientras que el resto de algoritmos pueden encontrar o no la solución.

Explicación de las estadísticas y trazas de algoritmos:

Experimentos Hill-Climbing:

```
===EXPERIMENTO 1. nQueens Hill Climbing. Coste medio de pasos en 1000 experimentos ===
Resultados de la búsqueda de 8 reinas con Hill Climbing:
Éxitos: 1419
Fallos: 8581
Porcentaje de éxito: 14.19%
Media de pasos en éxitos: 4.06
Porcentaje de fallos: 85.81%
Media de pasos en fallos: 3.04
Media de pasos (hasta un éxito): 22.44
Evaluaciones promedio: 242.707
===EXPERIMENTO 2. Random Restart Hill Climbing. Una ejecución ===
. Q . . . . . .
. . . . . Q . .
Q . . . . . .
. . . . . Q .
. . . Q . . . .
. . . . . Q
. . Q . . . . .
. . . . Q . . .
=== Resultados de Random Restart Hill Climbing ===
Solución encontrada: (1, 5, 0, 6, 3, 7, 2, 4)
Número de intentos: 9
Numero total de pasos (fallos + acierto): 24

===EXPERIMENTO 3. Random Restart Hill Climbing. Media de 1000 experimentos ===
Resultados de la búsqueda de 8 reinas con restart Hill Climbing:
Media de pasos: 23.50
Media de evaluaciones: 1788.84, debe ser cercano a (23.504+7) * 58 = 1769.23
```

Según la teoría, Hill Climbing, efectivamente encuentra solución alrededor de un 14% de las veces, y esta ejecución fue bastante acorde al deseado. De hecho el intento del Random Restart ha hecho sus fallos en muy pocos pasos, teniendo que tener una media de 28.

Uno de los problemas con los que me encontré en estos experimentos es la mala inicialización aleatoria del estado inicial, haciendo un shuffle del rango de valores del 1 al N, en vez de permitir que se repita la fila columna para las diferentes reinas. Y en el caso del Hill Climbing era más visual, bajando su rendimiento un 10% de éxito.

Experimentos Enfriamiento Simulado:

```
===EXPERIMENTO 4. 1000 experimentos enfriamiento simulado ===
Estadísticas sobre 1000 ejecuciones:
Tasa de éxito (0 conflictos): 34.00%
Media de conflictos finales: 0.76
Media de pasos en éxito: 563.01
Media de pasos en fracaso: 573.22
Media de pasos: 569.75
Media de evaluaciones: 2000.00

=== EXPERIMENTO 4-BIS. 1000 experimentos enfriamiento simulado ===
Estadísticas sobre 1000 ejecuciones:
Tasa de éxito (0 conflictos): 20.00%
Media de conflictos finales: 0.96
Media de pasos en éxito: 55.06
Media de pasos en fracaso: 60.23
Media de pasos: 59.20
Media de evaluaciones: 400.00
```

En el enfriamiento simulado vemos que las evaluaciones siempre son 2000 en cada ejecución, y eso es debido al limite de iteraciones asignadas al scheduler de temperatura, ya que este algoritmo siempre ejecutará todas las iteraciones posibles(a no ser de que llegue a un estado que no tenga acciones a realizar). Y es el doble debido a una pobre implementación del algoritmo, que llama dos veces a la función de evaluar el valor de un cierto estado. Vemos que su éxito es el doble que el Hill-Climbing, y que normalmente las soluciones finales solamente tienen 1 par de reinas que están en conflicto (el 0.76 < 1), aunque hay veces que hay más pares en conflicto, demostrado al no ser directamente el porcentaje de fallos (0.66). Su media de pasos para la solución final es brutal, y eso es debido a que la probabilidad de realizar una acción aleatoria peor es grande (la temperatura inicial es alta, y disminuye lentamente) y que el número de iteraciones del scheduler es también muy alto, permitiendo al algoritmo dar muchas vueltas entre estados (realizar mala acción, repararla). El porcentaje de éxito tampoco es muy grande ya que no deja de ser un Hill-Climbing, y por tanto se sigue atascando en máximos locales, mesetas y otras situaciones similares, teniendo que encadenar demasiadas acciones aleatorias peores con una probabilidad baja para salir de dichas situaciones.

En el experimento BIS, he puesto otro Scheduler de temperatura, que permite con menor probabilidad la toma de acciones peores, reduciendo su probabilidad de éxito, pero a su vez aumentando la optimalidad de las soluciones, y reduciendo el número de llamadas a la función de evaluación.

Experimentos Algoritmo Genético:

```
===EXPERIMENTO 5. Genéticos una ejecución ===
Mejor individuo encontrado: [5, 3, 6, 0, 7, 1, 4, 2]
Conflictos del estado 0
Es solución (IsGoal): True
Algoritmo Genético: 1577488 evaluaciones | Éxito: True

===EXPERIMENTO 6. 100 experimentos Genéticos ===
Media de evaluaciones en 100 experimentos: 1099505.05
Porcentaje de éxitos en 100 experimentos 98.0%
```

Los algoritmos genéticos confirman lo explicado anteriormente, su porcentaje de éxito es muy superior al resto, pero a cambio tardan mucho tiempo de ejecución, con 1 millón de evaluaciones de media por problema (evaluando más si han fallado, y menos si han acertado)

3. Algoritmos en situaciones de MultiAgentes (Juegos, 2 agentes)

En este apartado se han instrumentalizado los algoritmos de búsqueda para: obtener un valor, mediante una cierta función, de cualquier nodo de estado intermedio (sin que tenga que ser un estado terminal). Como los algoritmos están implementados de manera recursiva, para no tener problemas cambiando la implementación entera, los contadores nodos evaluados son variables globales que se incrementan por cada ejecución de sus respectivas funciones de max_value y min_value.

La función heurística es curiosa, ya que fuerza a la IA a jugar en columnas centrales, hasta que no detecta que hacerlo supondría tu victoria, y dependiendo de los pasos por delante que se le permite observar, captará o no estrategias que puedas usar como humano. La heurística por sí sola no le otorga estrategias reales, eso lo hace viendo un cierto número de pasos a futuro y realizando acciones beneficiosas para él, pero perjudiciales para el contrincante.

En principio, tanto la poda alpha-beta, como el minimax, con el mismo límite de profundidad deben alcanzar las mismas decisiones, solo que en menor tiempo para alpha-beta al podar las ramas que no puede tomar por ser peores que otras exploradas anteriormente.

3.1. La traza de ejecución para un problema conecta-4:

```
================================================================================================================
COMPARATIVA DE RENDIMIENTO: MINIMAX CUTOFF vs PODA ALFA-BETA CUTOFF
================================================================================================================
Profundidad (d) | MM Nodos    | MM Tiempo (s) | AB Nodos  | AB Podas  | AB Tiempo (s) | Ahorro Nodos (%)
----------------------------------------------------------------------------------------------------------------
1               | 6           | 0.0000        | 6         | 0         | 0.0000        | 0.0%
2               | 42          | 0.0000        | 27        | 3         | 0.0000        | 35.7%
3               | 258         | 0.0010        | 97        | 15        | 0.0010        | 62.4%
4               | 1554        | 0.0055        | 320       | 58        | 0.0020        | 79.4%
5               | 9330        | 0.0402        | 1013      | 202       | 0.0041        | 89.1%
6               | 55986       | 0.2274        | 3115      | 666       | 0.0144        | 94.4%
7               | 335922      | 1.3816        | 15705     | 3164      | 0.0691        | 95.3%
================================================================================================================
```

A continuación se muestra una traza de una partida mía contra la IA a profundidad 4, y observamos algo curioso en los últimos estados de la partida (no se ve muy claro en texto):

En cualquier momento, si una pieza se coloca en la columna 4, la victoria sería mía, al tener 3 combinaciones ganadoras posibles en filas incrementales a partir de la 2 (filas 2, 3, 4)

>> IA colocó ficha en la columna 6 (Nodos: 51, Podas: 9, Tiempo: 0.001s)
==========
.	X	O	.	X	.
.	X	X	.	O	.
.	X	X	.	X	.
.	**O**	X	**.**	X	X
.	X	**O**	**.**	**O**	**O**

| O | **O** | X | **.** | X | O |
| **O** | O | X | . | **O** | O |
==========
 1 2 3 4 5 6
Turno de O. Elige columna [1, 4, 6]: 4
==========
.	X	O	.	X	.
.	X	X	.	O	.
.	X	X	.	X	.
.	O	X	.	X	X
.	X	O	.	O	O
O	O	X	.	X	O
O	O	X	O	O	O
==========
 1 2 3 4 5 6
Turno de IA (X) calculando a profundidad d=4...
>> IA colocó ficha en la columna 6 (Nodos: 54, Podas: 13, Tiempo: 0.001s)
==========
.	X	O	.	X	.
.	X	X	.	O	.
.	X	X	.	X	X
.	O	X	.	X	X
.	X	O	.	O	O
O	O	X	.	X	O
O	O	X	O	O	O
==========
 1 2 3 4 5 6
Turno de O. Elige columna [1, 4, 6]: 4
==========
.	X	O	.	X	.
.	X	X	.	O	.
.	X	X	.	X	X
.	O	X	.	X	X
.	X	O	.	O	O
O	O	X	O	X	O
O	O	X	O	O	O
==========
 1 2 3 4 5 6
¡Gana O (Humano)!

Lo que se puede observar es que la IA, a pesar de que no pueda hacer nada, aún así, sigue evitando colocar una pieza en la columna 4, ya que la poda alfa-beta corta directamente la evaluación en la primera pieza colocada en esa columna, y por tanto sigue explorando otras opciones, la IA no sabe que yo ganaría todavía. De hecho lo evita incluso aunque no colocarla allí supondría que ganaría en ese mismo turno, por el mismo motivo.

La implementación del juego interactivo está hecha para IA vs IA, IA vs Humano con cualquiera empezando primero, y se podría realizar con una modificación para Humano vs Humano, aunque pierde el sentido en esta práctica (no evaluamos algoritmos de esa manera).
Un único detalle es que el valor de perder o ganar la partida debe ser el máximo o mínimo, comparado con los valores obtenidos de la función de evaluación para un estado, para que el algoritmo al encontrar que pierde evite por completo esa ruta.

Ya por curiosidad, el algoritmo que empieza la partida (jugando dos IAs) siempre gana o empata, excepto en el caso de profundidad 5, aunque no sabría explicar por qué.